

Вступ до JavaScript

(частина 2)

План

Оператори JavaScript (арифметичні оператори, оператори присвоєння, оператори порівняння, логічні (або реляційні) оператори, побітові оператори.

Перетворення типів

Умови (conditional statements)

Оператори

Оператори JavaScript

Оператор — це спеціальний символ, який повідомляє транслятору, що потрібна операція з деякими операндами.

Візьмемо простий вираз **2 * 3** дорівнює 6.

Тут 2 і 3 називають операндами (**аргументом/параметром**), а «*****» називають **оператором**.

Для роботи зі змінними та їх значеннями JavaScript підтримує всі стандартні операції (оператори/дії), більшість з яких присутні також у інших мовах програмування.

Оператори JavaScript

Унарний — це операція, яка застосовується до одного операнда. Наприклад, унарна операція мінус змінює знак числа:

```
const k = 5;  
console.log(-k);           // -5, unary minus  
console.log(-(k * 2));     // -10, unary minus applied to the result of  
                           // multiplication k * 2  
console.log(-(-3));        // 3
```

Бінарний — це операція, яка застосовується до двох операндів. Операція мінус також існує у бінарній формі:

```
const k = 6;  
const l = 2;  
console.log(k - l);        // 4, binary minus
```

Оператори JavaScript

JavaScript підтримує такі **типи операторів**:

- Арифметичні оператори
- Оператори присвоєння
- Оператори порівняння
- Логічні (або реляційні) оператори
- Побітові оператори
- Умовний (або потрійний) оператор

Арифметичні оператори JavaScript

Arithmetic (mathematical) operations:

Addition + (додавання)

```
const z = 20;  
console.log(z + 10); // 30
```

Subtraction – (віднімання)

```
const z = 8;  
console.log(z - 5); // 3
```

Multiplication * (множення)

```
const z = 5;  
console.log(z * 3); // 15
```

Division / (ділення)

```
const z = 12;  
console.log(z / 3); // 4
```

The **remainder of the division** (division modulo) **%** (остача від ділення)

```
console.log(8 % 4); // 0  
console.log(7 % 3); // 1  
console.log(8 % 3); // 2
```

Арифметичні оператори JS. Інкремент

Інкремент ++ - **збільшує** значення змінної **на одиницю**. Існує префіксний інкремент, який спочатку збільшує змінну на одиницю, а потім повертає її значення та постфіксний інкремент, яке спочатку повертає значення змінної, а потім збільшує його на одиницю:

```
let m = 5;  
let n = ++m; // prefix increment (first m increases by 1, then n=m)  
console.log(m); // 6  
console.log(n); // 6
```

```
let k = 5;  
let l = k++; // postfix increment (first l=k, then k increases by 1)  
console.log(k); // 6  
console.log(l); // 5
```

```
a = a + 1; // a++
```


Арифметичні оператори JS. Декремент

Декремент -- **зменшує** значення змінної **на одиницю**. Також існує префіксний декремент, який спочатку зменшує змінну на одиницю, а потім повертає її значення та постфіксний декремент, яке спочатку повертає значення змінної, а потім зменшує його на одиницю:

```
let m = 5;
let n = --m;    // prefix decrement (first m decreases by 1, then n=m)
console.log(m); // 4
console.log(n); // 4

let k = 5;
let l = k--;    // postfix decrement (first l=k, then k decreases by 1)
console.log(k); // 4
console.log(l); // 5
```

Exponentiation ** (піднесення до степеня)

Оператор піднесення до степеня `**` був доданий до мови JS порівняно недавно (ES2016).

Для цілого додатного числа b , результат $a ** b$ є a помноженим на себе b разів.

Наприклад:

```
console.log( 2 ** 2 ); // 4 (2 * 2)
```

```
console.log( 2 ** 3 ); // 8 (2 * 2 * 2)
```

```
console.log( 2 ** 4 ); // 16 (2 * 2 * 2 * 2)
```

Оператор також працює для нецілих чисел.

Наприклад:

```
console.log( 4 ** (1/2) ); // 2 (1/2 degree is equivalent to taking the square root)
```

```
console.log( 8 ** (1/3) ); // 2 (degree 1/3 is equivalent to taking a cubic root)
```

Пріоритетність операторів

Якщо вираз містить більше одного оператора, порядок виконання визначається їхнім **пріоритетом** або, іншими словами, порядком пріоритету операторів за замовчуванням.

Щоб змінити стандартний хід операцій, деякі вирази можна **помістити (взяти) у дужки**:

```
let m = 2;
```

```
let n = m * (4 + 2);
```

```
console.log(n); // 12
```

Precedence	Name	Sign
...	...	...
16	unary plus	+
16	unary negation	-
14	multiplication	*
14	division	/
13	addition	+
13	subtraction	-
...	...	...
3	assignment	=
...	...	...

Повна таблиця пріоритетності операторів:

https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Operators/Operator_Precedence#table

String addition

Операцію бінарного додавання можна використовувати для об'єднання кількох рядків в один:

```
let z = "coll";  
let y = "ege";  
let res = z + y;  
console.log(res);           // "college"
```

Якщо один з аргументів операції бінарного додавання є рядком, то другий буде перетворено в рядок:

```
console.log(5 + "1");       // "51"  
console.log("5" + 1);       // "51"  
console.log("10" + true);   // "10true"  
console.log("10" + null);   // "10null"  
let z;  
console.log("1" + z);       // "1undefined"
```

Інші математичні операції працюють тільки з числами і завжди перетворюють аргументи в числа:

```
console.log("5" - 2); // 3  
console.log(8 / "4"); // 2
```

Assignment Operator

Оператор присвоєння позначається знаком «=»:

```
let z = 10 - 4;  
console.log(z); // 6
```

Він обчислює (оцінює) вираз, який знаходиться праворуч, і призначає результат змінній, яка вказана зліва. Вираз може бути досить складним і включати будь-які інші змінні:

```
let z = 1;
```

```
let k = 2;
```

```
z = z + k + 3; // First, a calculation will be performed using the current value of the  
               variable z, then the result will be saved in it (the old variable z will be erased)
```

```
console.log(z); // 6
```



The "=" operator returns a value

Оператор присвоєння. Modify-in-place

Якщо потрібно застосувати операцію до змінної та одночасно зберегти в ній результат, то можна скоротити запис за допомогою **комбінованих операцій**:

Addition followed by assignment **+=**

```
let z = 5;  
z += 2; // z = z + 2;
```

Subtraction followed by assignment **-=**

```
let z = 5;  
z -= 2; // z = z - 2;
```

Multiplication followed by assignment ***=**

```
let z = 5;  
z *= 2; // z = z * 2;
```

Division followed by assignment **/=**

```
let z = 5;  
z /= 2; // z = z / 2;
```

Receiving the remainder of the division with subsequent assignment **%=**

```
let z = 5;  
z %= 2; // z = z % 2;
```

Оператори порівняння

Для перевірки умов використовуються **операції порівняння**. Усі операції порівняння є бінарними і результатом їх виконання є **логічний тип даних (Boolean)**:

Equals == if the arguments are equal - the result is true, otherwise false

```
console.log(10 == 10); // true
```

```
console.log(10 == 20); // false
```

Not equal != If the arguments are not equal, the result is true, otherwise false

```
console.log(10 != 10); // false
```

```
console.log(10 != 20); // true
```

Greater > if the left argument is greater than the right, the result is true, otherwise false

```
console.log(10 > 10); // false
```

```
console.log(10 > 20); // false
```

Less than < if the right argument is greater than the left - the result is true, otherwise false

```
console.log(10 < 10); // false
```

```
console.log(10 < 20); // true
```

Оператори порівняння

Greater than or equal to $\geq$, if the left argument is greater than the right or the arguments are equal, the result is true, otherwise false

```
console.log(10 >= 10); // true  
console.log(10 >= 20); // false
```

Less than or equal to $\leq$, if the right argument is greater than the left or the arguments are equal, the result is true, otherwise false

```
console.log(10 <= 10); // true  
console.log(10 <= 20); // true
```


Оператори порівняння. Порівняння рядків

Якщо обидва аргументи операцій порівняння є рядками, то відбувається порівняння **string-by-string comparison**:

```
console.log("K" < "L"); // true
```

Букви порівнюються за алфавітом. Чим далі стоїть буква в алфавіті, тим вона більша. Спочатку порівнюються перші літери, потім другі і так далі, поки одна не стане більшою за іншу.

Якщо перша буква лівого рядка більше, то лівий рядок більше, незалежно від решти символів:

```
console.log("SR" > "RS"); // true
```

Якщо перші літери в двох рядках однакові - порівняння відбувається за наступними літерами:

```
console.log("PRT" > "PRS"); // true, because "T" > "S"
```

Будь-яка літера більша за відсутність літери:

```
console.log("PRT" > "PR"); // true, because "T" more than nothing
```

Це порівняння називається лексикографічним.

Оператори порівняння. Строге порівняння

Звичайна перевірка рівності `==` має проблему. Вона не може відрізнити 0 від false. Це відбувається тому, що операнди різних типів перетворюються в числа за допомогою оператора рівності `==` :

```
console.log(false == 0); // true
console.log("1" == 1);   // true
console.log(1 == "01");  // true
console.log(true == 1);   // true
```

Якщо вам потрібно розрізняти подібні ситуації, використовуйте **strict equal** `===` або **strict not equal** `!==`. Вони порівнюють без перетворення типів даних, якщо типи аргументів різні, то результат відразу буде хибним:

```
console.log("1" === 1);    // false
console.log(false === 0);  // false
console.log("1" !== 1);    // true
```

Детальніше про перетворення типів:

<https://uk.javascript.info/type-conversions>

«Явне» та «Неявне» приведення типів:

<https://habr.com/ru/company/ruvds/blog/347866/>

Оператори порівняння. Порівняння з null та undefined

Звичайна перевірка рівності `==` має проблему. Вона не може відрізнити 0 від false. Це відбувається тому, що операнди різних типів перетворюються в числа за допомогою оператора рівності `==` :

```
console.log(false == 0); // true
console.log("1" == 1);  // true
console.log(1 == "01");  // true
console.log(true == 1);   // true
```

Якщо вам потрібно розрізняти подібні ситуації, використовуйте **strict equal** `===` або **strict not equal** `!==`. Вони порівнюють без перетворення типів даних, якщо типи аргументів різні, то результат відразу буде хибним:

```
console.log("1" === 1);    // false
console.log(false === 0);  // false
console.log("1" !== 1);    // true
```



Будь-які порівняння з undefined/null, окрім strict, слід проводити з обережністю.

Логічні оператори

Для поєднання кількох операцій порівняння використовують **логічні** оператори:

OR || (logical addition), if at least one of the arguments is true - the result is true, otherwise false

```
console.log(true || true); // true  
console.log(false || true); // true  
console.log(true || false); // true  
console.log(false || false); // false
```

AND && (logical multiplication), if at least one of the arguments is false - the result is false, otherwise true

```
console.log(true && true); // true  
console.log(false && true); // false  
console.log(true && false); // false  
console.log(false && false); // false
```

NOT ! (logical negation, inversion), reverses the value

```
console.log(!true); // false  
console.log(!false); // true
```

Логічні оператори

Приклади поєднання операцій порівняння та логічних операцій:

Чи знаходиться значення змінної **за межами діапазону** від 0 до 100:

```
let x = prompt("Enter your number", "");  
let res = x >= 0 && x <= 100;  
console.log(res);
```

Чи знаходиться значення змінної **в діапазоні** від 0 до 100 включно:

```
let x = prompt("Enter your number", "");  
let res = x < 0 || x > 100;  
console.log(res);
```

Логічні оператори. Короткий цикл обчислення

Кілька АБО обчислюються зліва направо. При цьому з метою економії ресурсів використовується **короткий цикл обчислення**. Припустимо, ви обчислюєте кілька АБО поспіль:

$$a || b || c || d$$

Якщо перший аргумент (a) істинний, то результат обов'язково буде істинним, а решта значень ігноруються

АБО обчислює рівно стільки раз, скільки необхідно перед для отримання першої істини.

Якщо true взагалі немає, повертається останній аргумент:

```
console.log(1 || 0 || 2); // 1
```

```
console.log(undefined || "Hi" || true); // "Hi"
```

```
console.log(0 || "" || false || null || undefined); // undefined
```

Логічні оператори. Короткий цикл обчислення.

Пріоритет

Операція AND також використовує короткий цикл обчислень, тільки замість true, AND обчислює значення **до першого false**:

```
console.log(1 && 0 && 2); // 0
console.log(0 && 1 && 2); // 0
console.log(1 && -1 && "Data" && true); // true
```

Пріоритет операції AND більше ніж OR:

```
console.log(2 || 1 && 0); // 2
```

Спочатку буде обчислено AND: 1 && 0, а потім OR: 2 || 0

Побітові оператори.

Побітові оператори працюють з 32-бітовими числами. Будь-який числовий операнд в операції перетворюється на 32-розрядне число. Результат перетворюється назад у число (number) JavaScript.

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
	OR	5 1	0101 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	Zero fill left shift	5 << 1	0101 << 1	1010	10
>>	Signed right shift	5 >> 1	0101 >> 1	0010	2
>>>	Zero fill right shift	5 >>> 1	0101 >>> 1	0010	2

Перетворення типів

Перетворення типів. String

Під час написання програм може знадобитися конвертувати дані з одного типу в інший. Три найбільш широко використовувані перетворення типів:

- to String
- to Number
- to Boolean

Перетворити в рядок:

- Function String()

```
let a = 20;  
let data = String(a);  
console.log(data);           // "20"  
console.log(typeof data);    // "string"
```
- Operation + and empty string

```
let a = 20;  
let data = a + "";  
console.log(data);           // "20"  
console.log(typeof data);    // "string"
```

Перетворення типів. Number

Convert to Number:

Function **Number()**

```
let a = "20";  
let num = Number(a);  
console.log(num);           // 20  
console.log(typeof num);    // "number"
```

- **Unary operation +**

```
let a = "20";  
let num = +a;  
console.log(num);           // 20  
console.log(typeof num);    // "number"
```

Перетворення типів. Number

Додаткові корисні функції для роботи з числами:

Function **parseInt()** - converts to an integer

```
console.log(parseInt("7")); // 7
console.log(parseInt("7.5")); // 7
console.log(parseInt("7nm")); // 7
console.log(parseInt("nm")); // NaN
```

- Function **parseFloat()** - converts to a fractional number

```
console.log(parseFloat("7")); // 7
console.log(parseFloat("7.125")); // 7.125
console.log(parseFloat("7nm")); // 7
console.log(parseFloat("nm")); // NaN
```

- Function **isNaN()** - checks if a value represents a number

```
console.log(isNaN(1)); // false
console.log(isNaN("1")); // false
console.log(isNaN("1m")); // true
console.log(isNaN(true)); // false
console.log(isNaN(null)); // false
console.log(isNaN(undefined)); // true
```

Перетворення типів. Boolean

Boolean conversion:

- Function **Boolean()**

```
let a = "1";  
let bln = Boolean(a);  
console.log(bln); // true  
console.log(typeof bln); // "boolean"
```

- Double Boolean **NOT !!**

```
let a = "1";  
let bln = !!a;  
console.log(bln); // true  
console.log(typeof bln); // "boolean"
```

Детальніше про перетворення типів:

<https://uk.javascript.info/type-conversions>

«Явне» та «Неявне» приведення типів:

<https://habr.com/ru/company/ruvds/blog/347866/>

УМОВИ (Conditional Statements)

Conditional Statements

Іноді нам потрібно виконувати різні дії залежно від умов. Для цього можна використовувати умовні оператори у своєму коді. У JavaScript наявні такі умовні оператори:

- використання **if**, щоб вказати блок коду, який буде виконано, якщо зазначена умова істинна;
- використання **if else**, щоб вказати блок коду, який буде виконано, якщо та сама умова є помилковою;
- використання **if else if**, щоб вказати нову умову для перевірки, якщо перша умова хибна;
- використання **switch**, щоб вказати багато альтернативних блоків коду для виконання.

if statement

Оператор **if** є основним оператором управління, який дозволяє JavaScript приймати рішення та виконувати код з залежності від умови.

Синтаксис основного оператора if:

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Після ключового слова if стоїть умова в дужках. **Умова** — це будь-який вираз, який в кінцевому підсумку повертає логічний тип. Якщо умова **істинна**, то виконується код з **блоку if**, приклад:

```
const a = 11;  
if (a > 0) {  
    console.log("Number " + a + " positive");  
}
```


if statement. Boolean conversion

Оператор **if (...)** оцінює вираз у дужках і перетворює результат у логічне значення (булевий тип).

Правила перетворення:

- число 0, порожній рядок "", null, undefined і NaN стають хибними.
- інші значення стають істинними.

Приклад:

```
if (5) {  
    console.log("Always running");  
} else {  
    console.log("Never done");  
}
```

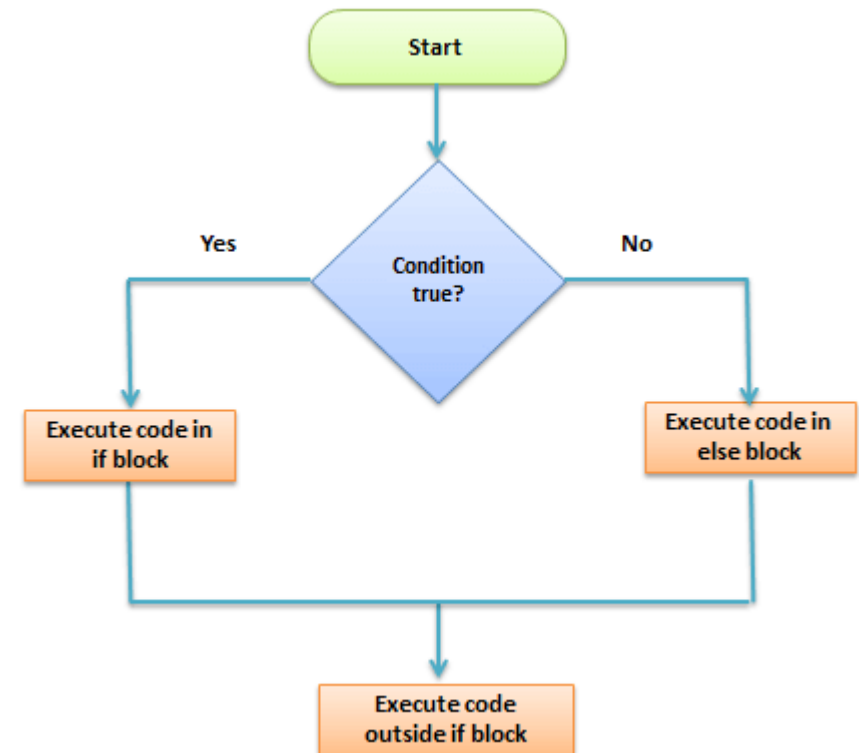
If..else statement.

Оператор **'if..else'** є наступною формою оператора керування, який дозволяє JavaScript виконувати інструкції більш контрольованим способом.

```
if (condition) {  
  // block of code to be executed  
  if the condition is true  
  
} else {  
  // block of code to be executed  
  if the condition is false  
}
```

Example:

```
const a = 11;  
if (a > 0) {  
  console.log("Number " + a + " positive");  
} else {  
  console.log("Number " + a + " negative");  
}
```



If..else..if statement.

Оператор **'if..else..if'** є наступною формою оператора керування, який дозволяє JavaScript виконувати інструкції більш контрольованим способом.

```
if (condition1) {  
    // block of code to be executed if the condition1 is true  
}  
else if (condition2) {  
    // block of code to be executed if the condition2 is true  
}  
else {  
    // block of code to be executed if no conditions is true  
}
```

Example:

```
const a = 11;  
if (a > 0) {  
    console.log("Number " + a + " positive");  
} else if (a < 0) {  
    console.log("Number " + a + " negative");  
} else {  
    console.log("Number " + a + " is exactly zero");  
}
```

Умовний оператор (тернарний оператор)

Умовний оператор представлений знаком питання **?**. Його також називають «тернарний», оскільки цей оператор, єдиний у своєму роді, має три аргументи

Цей оператор спочатку оцінює вираз з точки зору істинного чи хибного значення, а потім виконує один із двох заданих операторів залежно від результату оцінки.

```
let result = умова ? значення1 : значення2;
```

Якщо умова істинна? Тоді повертається значення1 : інакше повертається значення2
приклад:

```
let age = 21;  
let accessAllowed = (age > 18) ? true : false;  
console.log(accessAllowed); // true
```

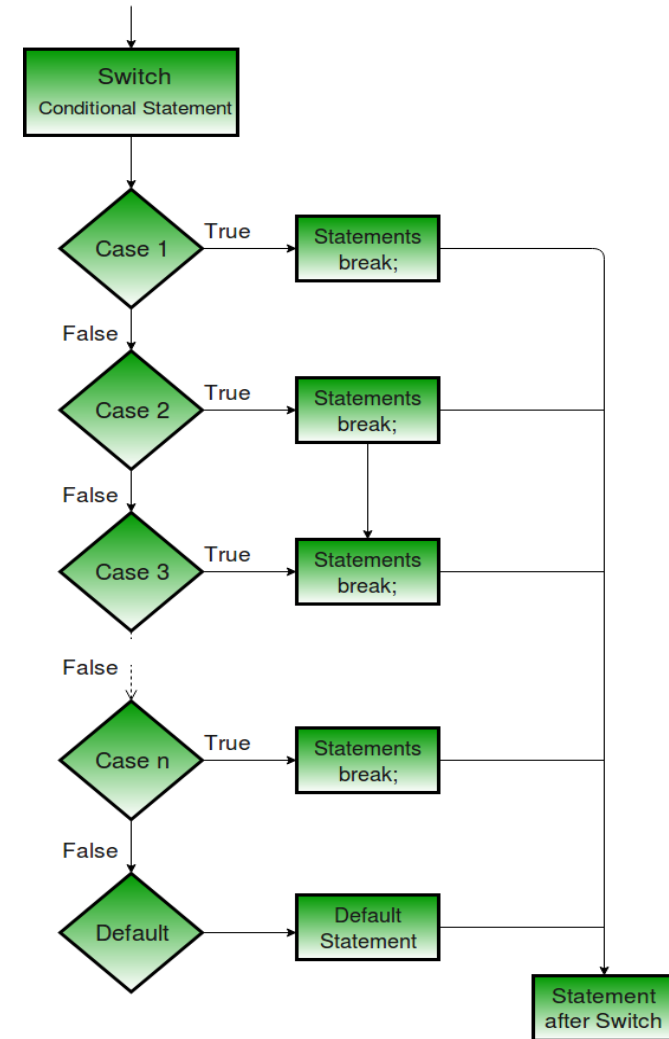
switch statement

Конструкція **switch** дозволяє вибрати реалізацію одного з багатьох блоків для виконання.

```
switch (expression) {  
    case 'value1':  
        // code block  
        break;  
    case 'value1':  
        // code block  
        break;  
    default:  
        // code block  
}
```

Ось як це працює:

- вираз **switch** обчислюється один раз.
- значення виразу **строго** порівнюється зі значеннями кожного випадку.
- якщо є збіг, виконується відповідний блок коду.



switch statement

Отже значення **expression** перевіряється на строгу рівність (===) значенню із першого блоку **case** (яке дорівнює value1), потім значенню із другого блоку (value2) і так далі.

Якщо строго рівне значення знайдено, то **switch** починає виконання коду із відповідного **case** до найближчого **break** або до кінця всієї конструкції **switch**.

Якщо жодне case-значення не збігається – виконується код із блоку **default** (якщо він присутній).

```
let a = 2 + 2;
switch (a) {
  case 3:
    alert( 'Замало' );
    break;
  case 4:
    alert( 'Точнісінько!' );
    break;
  case 5:
    alert( 'Забагато' );
    break;
  default:
    alert( 'Я не знаю таких значень' );
}
```

```
let userInput = prompt("Введіть тест або число:");
switch (true) {
  case isNaN(userInput):
    console.log("Не є числом");
    break;
  case userInput > 0:
    console.log("Додатне число");
    break;
  case +userInput === 0: // Конвертуємо в число та
    порівнюємо з 0
    console.log("0");
    break;
  case userInput < 0:
    console.log("Від'ємне число");
    break;
  default:
    console.log("Інший тип даних");
}
```

Підсумки

Вивчили оператори JavaScript (арифметичні оператори, оператори присвоєння, оператори порівняння, логічні (або реляційні) оператори, побітові оператори, умовний (або тернарний) оператор.

Вивчили правила перетворення типів

Навчилися програмувати розгалуження в JS (conditional statements)

Дякую за увагу...
